

COS 214 - Practical 5

u25463382 - Melaney Mutunhu

u25110782 - Robert Sibanda

u25657764 - Chesly Hwata

Github link - <https://github.com/Melcodes03/COS-214-Practical-5>

Task 1

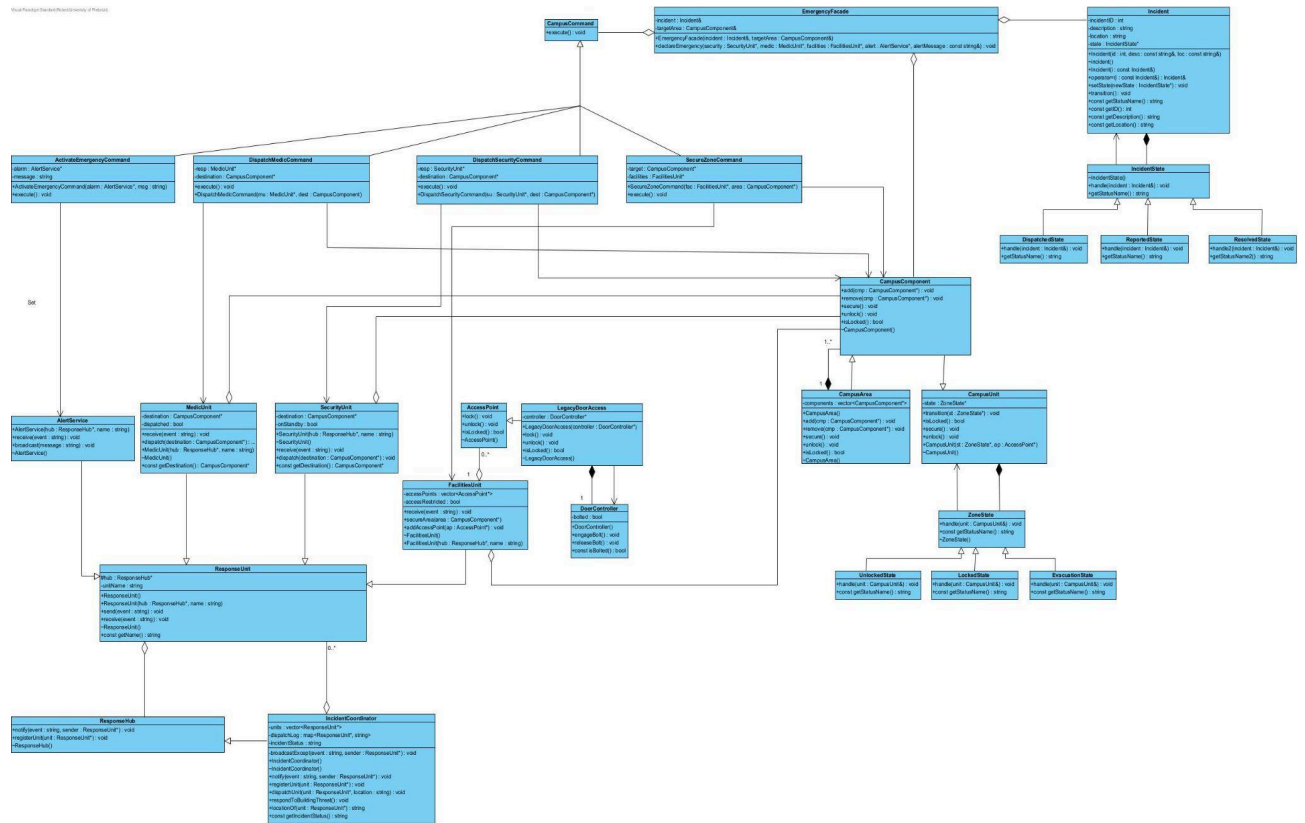
Short system description

CampusGuard is an emergency-response coordination platform for a large university campus. Staff can report an incident, dispatch response units, secure areas and alert the campus, and each incident moves through a defined life cycle from Reported to Dispatched.

Four response components (security, medical, facilities and campus alerts) collaborate through an IncidentCoordinator instead of calling each other directly. Operator actions such as dispatching a unit or securing an area are Command objects, physical doors are controlled through an adapter around a legacy door controller, and an EmergencyFacade runs the full emergency workflow in one call.

Two scenarios demonstrate the system: a fire alarm in the Engineering Building, run through the Facade, and suspicious activity in Residence Block C, run through direct commands.

UML Class diagram



GoF Mapping & Rationale

GoF Pattern	Role	Class in CampusGuard	Rationale
State (Incident)	Context	Incident	Holds an IncidentState* and delegates transition() to it, so behaviour changes as the incident moves through Reported, Dispatched, and Resolved without any if/switch on a status flag.
State (Incident)	State(abstract)	IncidentState	Declares handle() and getStatusName() so every phase of an incident can define its own behaviour under one common interface.
State	ConcreteState	ReportedState, DispatchedState, ResolvedState	Each one decides what

(Incident)		DispatchedState, ResolvedState	the next state should be, keeping the transition logic local to the state it belongs to instead of centralised in Incident.
State (Zone)	Context	CampusUnit	Holds a ZoneState* for its own lock lifecycle, kept separate from Incident's state since a room's lock status and an incident's operational status are different things.
State (Zone)	State (abstract)	ZoneState	Declares handle() and getStatusName(), the same idea as IncidentState but for a different lifecycle.
State (Zone)	ConcreteState	LockedState, UnlockedState, EvacuationState	Each one defines what happens to the unit in that access state.
Facade	Facade	EmergencyFacade	Provides one method, declareEmergency(), that coordinates dispatching security, dispatching medics, securing the area, and sending the alert, all in one call.
Facade	Subsystem classes	SecurityUnit, MedicUnit, FacilitiesUnit, AlertService, Incident	Each still works independently and can be called directly when the full orchestrated workflow isn't needed.
Composite	Component (abstract)	CampusComponent	Declares add(), remove(), secure(), unlock(), isLocked(), so a client can treat one unit and a whole structural group the same way.
Composite	Leaf	CampusUnit	A single physical unit (e.g. a room) with no children, implements the interface directly.

Composite	Composite	CampusArea	Holds a list of CampusComponent* and delegates secure()/unlock()/isLocked() to each child, so locking a whole building works the same as locking one room.
Command	Command (abstract)	CampusCommand	Declares execute(), so a request is represented and stored as an object instead of called directly.
Command	ConcreteCommand	DispatchSecurityCommand, DispatchMedicCommand, SecureZoneCommand, ActivateEmergencyCommand	Each binds a specific receiver to a specific action, so whoever runs the command doesn't need to know what it actually does.
Command	Receiver	SecurityUnit, MedicUnit, FacilitiesUnit, AlertService	Performs the real work the command was created to trigger.
Command	Invoker	EmergencyFacade	Builds and executes the command objects as part of coordinating the emergency workflow.
Mediator	Mediator (interface)	ResponseHub	Declares notify() and registerUnit(), so colleagues can trigger coordinated behaviour without knowing about each other directly.
Mediator	ConcreteMediator	IncidentCoordinator	Implements notify() to decide what happens across response units when one of them reports an event.
Mediator	Colleague (abstract)	ResponseUnit	Holds a reference to the hub and provides send(), so any concrete colleague can report an event

			through the same interface.
Mediator	ConcreteColleague	SecurityUnit, MedicUnit, FacilitiesUnit, AlertService	Each calls send() on itself when something happens, and lets the coordinator decide what happens next.
Adapter	Target	AccessPoint	Defines the interface CampusGuard actually wants to call: lock(), unlock(), isLocked().
Adapter	Adaptee	DoorController	The legacy hardware interface, with different method names (engageBolt(), releaseBolt(), isBolted()) that don't match what CampusGuard expects.
Adapter	Adapter	LegacyDoorAccess	Translates calls from the AccessPoint interface into what DoorController actually expects.
Adapter	Client	FacilitiesUnit	Calls the AccessPoint interface only, through addAccessPoint(), unaware a legacy system sits behind it.

Justification for the two team selected patterns

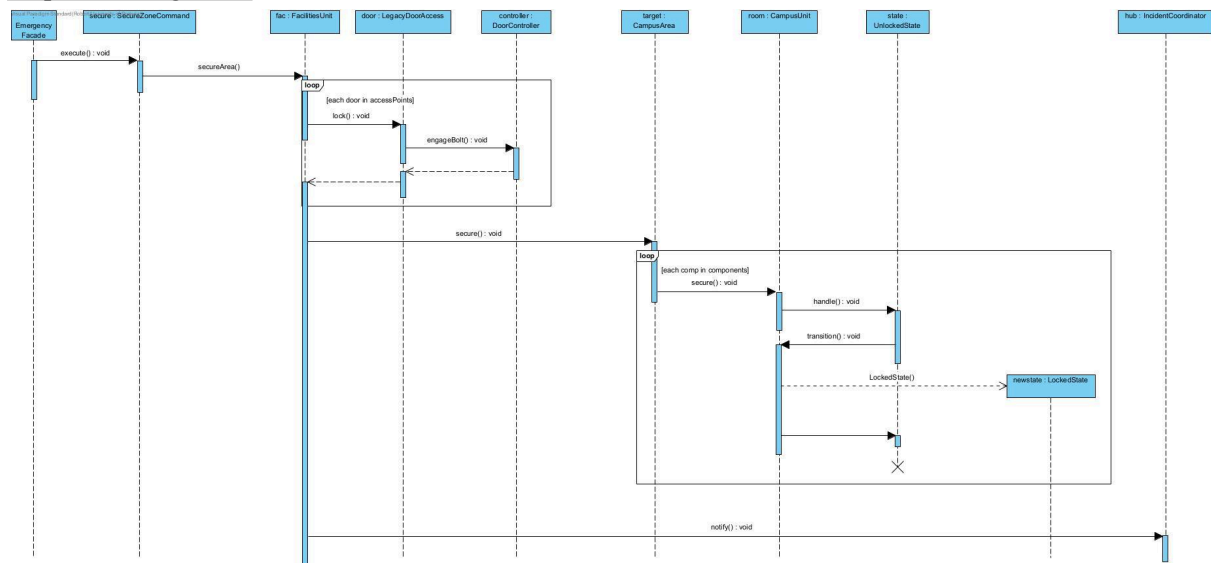
The team selected patterns are the State and the Composite design patterns. Within the specification for CampusGuard it is stated that the system must support “registering incidents and changing their operational status”. So for example if an emergency progresses, the rules for what actions are allowed to happen in the system would change completely. A solution that could work would be using a status enum and use large conditional blocks to check if an action is allowed, however this violates the rule that prohibits “giant switch statements, large centralised if/else chains, type checks, or manual client code that knows every concrete class“. The state pattern solves this problem by encapsulating the behaviour of each operational phase into its own class thereby making the code highly cohesive and open for extension.

It is also stated within the specification that CampusGuard must support “dispatching campus security, medical responders or facilities staff”, so the system needs a clean way to issue instructions to both individual units (like a single medic) and complex task forces that involve multiple departments without duplicating logic. The system also needs to handle “building access actions such as locking,

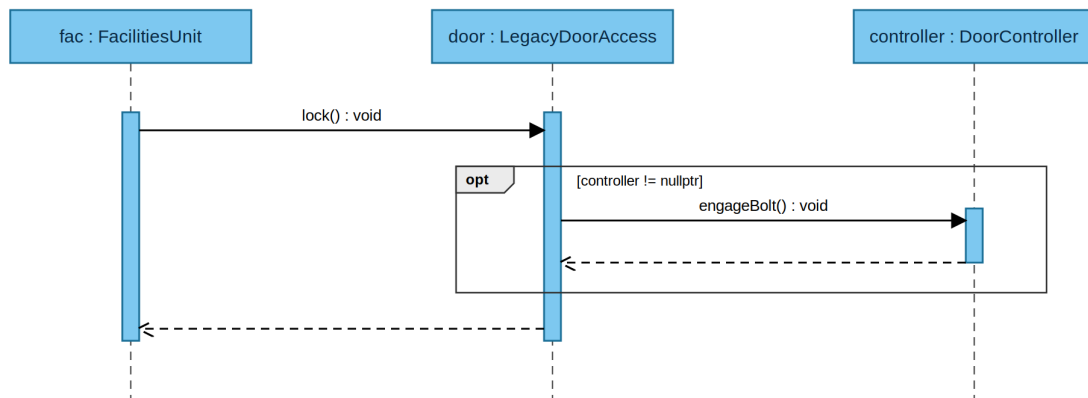
unlocking or restricting selected areas”. A university is naturally hierarchical in nature (for example campus -> buildings -> floors -> rooms), so if an operator needs to lock down a single specific lab instead of the entire engineering building the system needs a way to process these instructions without writing different logic for single rooms vs large structural groups. One method would be to use manual type checks before iterating over lists. However this violates the rule against “type checks or manual client code that knows every concrete class”. The Composite patterns provides a uniform interface so the client treats a single responder and a massive structural hierarchy the same way.

Task 4

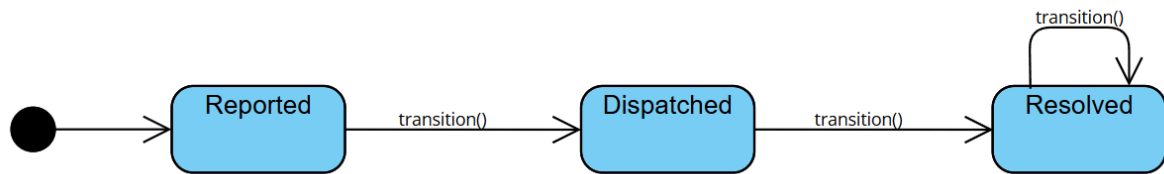
Sequence Diagram 1



Sequence Diagram 2



State Diagram



Task 5

Concise design and ownership rationale

- Composite pattern: The CampusArea class enforces a strict composition relationship over the CampusComponent. A CampusArea(1) maps to 0..* CampusComponent interfaces. Memory Ownership is top down in that when a parent CampusArea is destroyed, its destructor iterates through its vector and deletes all child pointers. This makes a cascading destruction of the entire building tree down to the leaf CampusUnit nodes.
- State pattern: The system uses 2 state hierarchies. CampusUnit owns a ZoneState and Incident owns an IncidentState. The context classes hold exclusive ownership (Composition relationship) of these dynamically allocated state objects and during a state transition the context deletes its current state pointer before allocating the new state so that there are no dangling pointers left on the heap while contexts move through their lifecycles. The final state is destroyed within the context's own destructor.
- Mediator pattern: The IncidentCoordinator acts as the central hub that facilitates communication in the system. It maintains a 1 to 0..* aggregation relationship with ResponseUnit colleagues. IncidentCoordinator stores pointers to the ResponseUnits to route broadcasts but it explicitly does not own them. The responders and the coordinator are instantiated independently and must be destroyed independently by the client to prevent double free errors.
- Adapter and command: FacilitiesUnit has a 1 to 0..* aggregation relationship with AccessPoint adapters to interface with the legacy hardware. The concrete CampusCommand objects hold 1 to 1 references to their receivers and deployment destinations. The commands and the facilities unit don't take explicit ownership of these pointers.

Where the 6 patterns interact

- The Facade (EmergencyFacade) serves as the central client interface. When an emergency is declared it will bind the physical destination from Composite(CampusComponent) to an incident managed by the state pattern(IncidentState)

- In order to decouple the request from the execution, the facade instantiates concrete command objects (like SecureZoneCommand). These commands target the specific Composite are and are routed to the appropriate responder units
- When the FacilitiesUnit receives the command it utilizes the Adapter pattern(LegacyDoorAccess) to translate the modern system request into instructions that physically lock the Legacy DoorController hardware.
- Simultaneously, the command triggers the logical lockdown on the root of the Composite tree. This instruction moves recursively down the hierarchy till it reaches CampusUnit(the leaf nodes) which makes them update their internal state (ZoneState) from an unlocked phase to a locked one.
- Once the physical hardware is secured through the adapter and the logical building layout has been updated through the tree, the FacilitiesUnit reports its success back to IncidentCoordinator(Mediator). The central hub then broadcasts the status update across the entire responder network which ensures all units remain synchronized as the facade transitions the overall emergency lifecycle to its next lifecycle state.

GDB investigation and Valgrind evidence

Symptom

While running Scenario 3 ("Escalation and System Stand Down"), the zone state of a secured unit was expected to escalate from Locked to Evacuation on a second secure() call. main.cpp calls this twice back to back with a comment stating the intent:

```
cpp
// test to evacuation state
std::cout << "\n-- Testing Zone State Escalation --" << std::endl;
blockC.secure();
blockC.secure();
```

The escalation never happens. No unit in Block C ever reaches the Evacuation state, no matter how many times secure() is called on it.

Cause

The bug lives in CampusUnit::secure():

```
cpp
void CampusUnit::secure() {
    if (this->state != nullptr && this->state->getStatusName() == "Unlocked") {
        this->state->handle(*this);
    }
}
```

handle() is only invoked when the current state's name is exactly "Unlocked". This correctly drives the first transition (Unlocked -> Locked), but once the unit reaches "Locked", the condition is permanently false for every future call, so handle() is never invoked again. LockedState::handle()

(which performs the Locked -> Evacuation transition) becomes unreachable through the public secure() interface, even though the state hierarchy itself supports the transition.

Debugging Evidence (GDB)

A breakpoint was set on CampusUnit::secure() and the program was run under GDB.

```
(gdb) break CampusUnit::secure
(gdb) run
```

The program ran through Scenario 1 and Scenario 2, securing units normally. Breakpoint 1 was disabled for those routine hits with ignore 1 3 to skip ahead to Scenario 3's escalation test, where a CampusUnit in blockC was already Locked from Scenario 2's secureBlockC.execute():

Breakpoint 1, CampusUnit::secure (this=0x555555581950) at CampusUnit.cpp:22

```
22 void CampusUnit::secure() {
(gdb) print this->state->getStatusName()
$1 = "Locked"
```

Stepping to the guard condition confirmed the state was still "Locked", not "Unlocked":

```
(gdb) next
23 if (this->state != nullptr && this->state->getStatusName() == "Unlocked") {
(gdb) print this->state->getStatusName()
$2 = "Locked"
```

Stepping again skipped straight past the handle() call to the end of the function, since the condition was false, and the state was unchanged:

```
(gdb) next
26 }
(gdb) print this->state->getStatusName()
$3 = "Locked"
```

The breakpoint was then disabled and the program allowed to run to completion. The full remaining console output was inspected for LockedState::handle()'s message ("...forcing unlock for evacuation"), which never appeared anywhere in the run:

```
(gdb) disable 1
(gdb) continue
Continuing.
```

-- Testing Hardware and Logical Unlocking --
[DoorController] bolt released.

-- Testing Mediator Threat and Stand Down Broadcasts --
[IncidentCoordinator] THREAT DETECTED. Triggering response protocols.
[Medic-1] standing down.
[Facilities-1] restoring normal access.

[CampusAlertSystem] campus-wide alert: Incident resolved. Normal operations resumed.
[IncidentCoordinator] status is now RESOLVED after event "INCIDENT_RESOLVED" from SecurityGuard-1

-- Testing Composite Memory Management --
[Inferior 1 (process 1391) exited normally]

This confirmed the root cause directly: the unit remained "Locked" for the rest of the program's execution. The Evacuation state, and its associated behaviour, was never reached.

Correction

The guard in CampusUnit::secure() should not restrict handle() to only the "Unlocked" state. The State pattern is meant to let each concrete state decide its own next transition; re-checking a specific state name in the client-facing method defeats that and hard-codes only the first transition.

Before:

```
cpp
void CampusUnit::secure() {
    if (this->state != nullptr && this->state->getStatusName() == "Unlocked") {
        this->state->handle(*this);
    }
}
```

After:

```
cpp
void CampusUnit::secure() {
    if (this->state != nullptr) {
        this->state->handle(*this);
    }
}
```

This lets UnlockedState::handle() still move to Locked, LockedState::handle() move to Evacuation, and EvacuationState::handle() remain a no op terminal state, exactly as each concrete state already implements no string comparison needed in CampusUnit itself.

```
hp@DESKTOP-8KLOCR6:/mnt/c/Users/hp/OneDrive - University of Pretoria/Documents/GitHub/COS-214-Practical-5$ make clean
make all
rm -f ActivateEmergencyCommand.o AlertService.o CampusArea.o CampusComponent.o CampusUnit.o DispatchMedicCommand.o DispatchSecurityCommand.o Dispatch
edState.o DoorController.o EmergencyFacade.o EvacuationState.o FacilitiesUnit.o Incident.o IncidentCoordinator.o LegacyDoorAccess.o LockedState.o Med
icUnit.o ReportedState.o ResolvedState.o ResponseUnit.o SecureZoneCommand.o SecurityUnit.o UnlockedState.o main.o CampusGuard
g++ -std=c++11 -Wall -g -c -o ActivateEmergencyCommand.o ActivateEmergencyCommand.cpp
g++ -std=c++11 -Wall -g -c -o AlertService.o AlertService.cpp
g++ -std=c++11 -Wall -g -c -o CampusArea.o CampusArea.cpp
g++ -std=c++11 -Wall -g -c -o CampusComponent.o CampusComponent.cpp
g++ -std=c++11 -Wall -g -c -o CampusUnit.o CampusUnit.cpp
g++ -std=c++11 -Wall -g -c -o DispatchMedicCommand.o DispatchMedicCommand.cpp
g++ -std=c++11 -Wall -g -c -o DispatchSecurityCommand.o DispatchSecurityCommand.cpp
g++ -std=c++11 -Wall -g -c -o DispatchedState.o DispatchedState.cpp
g++ -std=c++11 -Wall -g -c -o DoorController.o DoorController.cpp
g++ -std=c++11 -Wall -g -c -o EmergencyFacade.o EmergencyFacade.cpp
g++ -std=c++11 -Wall -g -c -o EvacuationState.o EvacuationState.cpp
g++ -std=c++11 -Wall -g -c -o FacilitiesUnit.o FacilitiesUnit.cpp
g++ -std=c++11 -Wall -g -c -o Incident.o Incident.cpp
g++ -std=c++11 -Wall -g -c -o IncidentCoordinator.o IncidentCoordinator.cpp
g++ -std=c++11 -Wall -g -c -o LegacyDoorAccess.o LegacyDoorAccess.cpp
g++ -std=c++11 -Wall -g -c -o LockedState.o LockedState.cpp
g++ -std=c++11 -Wall -g -c -o MedicUnit.o MedicUnit.cpp
g++ -std=c++11 -Wall -g -c -o ReportedState.o ReportedState.cpp
g++ -std=c++11 -Wall -g -c -o ResolvedState.o ResolvedState.cpp
g++ -std=c++11 -Wall -g -c -o ResponseUnit.o ResponseUnit.cpp
g++ -std=c++11 -Wall -g -c -o SecureZoneCommand.o SecureZoneCommand.cpp
g++ -std=c++11 -Wall -g -c -o SecurityUnit.o SecurityUnit.cpp
g++ -std=c++11 -Wall -g -c -o UnlockedState.o UnlockedState.cpp
g++ -std=c++11 -Wall -g -c -o main.o main.cpp
g++ -std=c++11 -Wall -g -o CampusGuard ActivateEmergencyCommand.o AlertService.o CampusArea.o CampusComponent.o CampusUnit.o DispatchMedicCommand.o D
ispatchSecurityCommand.o DispatchedState.o DoorController.o EmergencyFacade.o EvacuationState.o FacilitiesUnit.o Incident.o IncidentCoordinator.o Leg
acyDoorAccess.o LockedState.o MedicUnit.o ReportedState.o ResolvedState.o ResponseUnit.o SecureZoneCommand.o SecurityUnit.o UnlockedState.o main.o
```

```
hp@DESKTOP-8KLOCR6:/mnt/c/Users/hp/OneDrive - University of Pretoria/Documents/GitHub/COS-214-Practical-5$ gdb ./CampusGuard
```

```
GNU gdb (Ubuntu 17.1-2ubuntu1) 17.1
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.
```

```
For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./CampusGuard...
(gdb) break CampusUnit::secure
Breakpoint 1 at 0x7ab7: file CampusUnit.cpp, line 22.
(gdb) run
Starting program: /mnt/c/Users/hp/OneDrive - University of Pretoria/Documents/GitHub/COS-214-Practical-5/CampusGuard
```

```
This GDB supports auto-downloading debuginfo from the following URLs:
<https://debuginfod.ubuntu.com>
Enable debuginfod for this session? (y or [n]) n
Debuginfod has been disabled.
To make this setting permanent, add 'set debuginfod enabled off' to .gdbinit.
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".
```

```
[SCENARIO 1: Fire alarm, Engineering Building]
Patterns: Facade, Command, Mediator, Composite, Adapter, State
```

```
Incident 1 reported at Engineering Building, Lab 4 -- status: Reported
```

```
[EmergencyFacade] declaring emergency for incident 1 (Fire alarm triggered)
[SecurityGuard-1] dispatched to site.
[Medic-1] awaiting access at site.
```

```

[SCENARIO 1: Fire alarm, Engineering Building]
Patterns: Facade, Command, Mediator, Composite, Adapter, State

Incident 1 reported at Engineering Building, Lab 4 -- status: Reported

[EmergencyFacade] declaring emergency for incident 1 (Fire alarm triggered)
[SecurityGuard-1] dispatched to site.
[Medic-1] awaiting access at site.
[Facilities-1] preparing access points for responders.
[IncidentCoordinator] status is now ACTIVE after event "SECURITY_DISPATCHED" from SecurityGuard-1
[Medic-1] dispatched to site.
[Facilities-1] preparing access points for responders.
[IncidentCoordinator] status is now ACTIVE after event "MEDIC_DISPATCHED" from Medic-1
[DoorController] bolt engaged.

Breakpoint 1, CampusUnit::secure (this=0x55555581020) at CampusUnit.cpp:22
22 void CampusUnit::secure() {
(gdb) ignore 1 3
Will ignore next 3 crossings of breakpoint 1.
(gdb) continue
Continuing.
Unit is unlocked, securing and moving to locked
[Facilities-1] area physically and logically secured.
[CampusAlertSystem] campus-wide alert: Affected area has been secured.
[IncidentCoordinator] status is now CONTAINED after event "AREA_SECURED" from Facilities-1
[CampusAlertSystem] campus-wide alert: Fire reported in Engineering Building, evacuate the area.
Incident 1 has been reported, moving to dispatched
[EmergencyFacade] incident 1 is now Dispatched

Physical door bolted: yes
Engineering building locked: yes
Incident status: Dispatched

[SCENARIO 2: Suspicious activity, Residence Block C]
Patterns: Command (direct, no Facade), Mediator, Composite (multi-unit), Adapter, State, handled invalid-operation case

Incident 2 reported at Residence Block C -- status: Reported
Block C locked before response: no

Patterns: Command (direct, no Facade), Mediator, Composite (multi-unit), Adapter, State, handled invalid-operation case

Incident 2 reported at Residence Block C -- status: Reported
Block C locked before response: no

[SecurityGuard-1] dispatched to site.
[Medic-1] awaiting access at site.
[Facilities-1] preparing access points for responders.
[IncidentCoordinator] status is now ACTIVE after event "SECURITY_DISPATCHED" from SecurityGuard-1
[DoorController] bolt engaged.
[DoorController] bolt engaged.
[DoorController] bolt engaged.
Unit is unlocked, securing and moving to locked
Unit is unlocked, securing and moving to locked
[Facilities-1] area physically and logically secured.
[CampusAlertSystem] campus-wide alert: Affected area has been secured.
[IncidentCoordinator] status is now CONTAINED after event "AREA_SECURED" from Facilities-1
[CampusAlertSystem] campus-wide alert: Security responding to Residence Block C, please remain in your rooms.

Block C locked after response: yes
Engineering door still bolted from Scenario 1: yes
Incident 2 has been reported, moving to dispatched
Incident status: Dispatched

Attempting to dispatch medic to a null destination...
Handled safely: null destination detected, medic's assignment left unchanged.

[SCENARIO 3: Escalation and System Stand Down]
Hitting remaining state transitions, unlock procedures, and coordinator branches

-- Testing Incident State Escalation --
Incident 2 units have been dispatched, marking as resolved
Incident 2 is already resolved, nothing more to do

-- Testing Zone State Escalation --

Breakpoint 1, CampusUnit::secure (this=0x55555581950) at CampusUnit.cpp:22
22 void CampusUnit::secure() {
(gdb) print this->state->getStatusName()

```

```

Breakpoint 1, CampusUnit::secure (this=0x55555581950) at CampusUnit.cpp:22
22 void CampusUnit::secure() {
(gdb) print this->state->getStatusName()
$1 = "Locked"
(gdb) next
23 if (this->state != nullptr && this->state->getStatusName() == "Unlocked") {
(gdb) print this->state->getStatusName()
$2 = "Locked"
(gdb) next
26 }
(gdb) print this->state->getStatusName()
$3 = "Locked"
(gdb) finish
Run till exit from #0 CampusUnit::secure (this=0x55555581950) at CampusUnit.cpp:26
CampusArea::secure (this=0x7ffffffdbf0) at CampusArea.cpp:22
22 for(const auto& comp : this->components){
(gdb) continue
Continuing.

Breakpoint 1, CampusUnit::secure (this=0x55555581930) at CampusUnit.cpp:22
22 void CampusUnit::secure() {
(gdb) disable 1
(gdb) continue
Continuing.

-- Testing Hardware and Logical Unlocking --
[DoorController] bolt released.

-- Testing Mediator Threat and Stand Down Broadcasts --
[IncidentCoordinator] THREAT DETECTED. Triggering response protocols.
[Medic-1] standing down.
[Facilities-1] restoring normal access.
[CampusAlertSystem] campus-wide alert: Incident resolved. Normal operations resumed.
[IncidentCoordinator] status is now RESOLVED after event "INCIDENT_RESOLVED" from SecurityGuard-1

-- Testing Composite Memory Management --
[Inferior 1 (process 1391) exited normally]
(gdb) quit

```

Valgrind

```

hp@DESKTOP-8KLOCR6:/mnt/c/Users/hp/OneDrive - University of Pretoria/Documents/Github/COS-214-Practical-5$ make valgrind
valgrind --leak-check=full --show-leak-kinds=all --track-origins=yes ./CampusGuard
==1550== Memcheck, a memory error detector
==1550== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==1550== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==1550== Command: ./CampusGuard
==1550==

[SCENARIO 1: Fire alarm, Engineering Building]
Patterns: Facade, Command, Mediator, Composite, Adapter, State

Incident 1 reported at Engineering Building, Lab 4 -- status: Reported

[EmergencyFacade] declaring emergency for incident 1 (Fire alarm triggered)
[SecurityGuard-1] dispatched to site.
[Medic-1] awaiting access at site.
[Facilities-1] preparing access points for responders.
[IncidentCoordinator] status is now ACTIVE after event "SECURITY_DISPATCHED" from SecurityGuard-1
[Medic-1] dispatched to site.
[Facilities-1] preparing access points for responders.
[IncidentCoordinator] status is now ACTIVE after event "MEDIC_DISPATCHED" from Medic-1
[DoorController] bolt engaged.
Unit is unlocked, securing and moving to locked
[Facilities-1] area physically and logically secured.
[CampusAlertSystem] campus-wide alert: Affected area has been secured.
[IncidentCoordinator] status is now CONTAINED after event "AREA_SECURED" from Facilities-1
[CampusAlertSystem] campus-wide alert: Fire reported in Engineering Building, evacuate the area.
Incident 1 has been reported, moving to dispatched
[EmergencyFacade] incident 1 is now Dispatched

Physical door bolted: yes
Engineering building locked: yes
Incident status: Dispatched

[SCENARIO 2: Suspicious activity, Residence Block C]
Patterns: Command (direct, no Facade), Mediator, Composite (multi-unit), Adapter, State, handled invalid-operation case

Incident 2 reported at Residence Block C -- status: Reported
Block C locked before response: no

```

```
[SecurityGuard-1] dispatched to site.
[Medic-1] awaiting access at site.
[Facilities-1] preparing access points for responders.
[IncidentCoordinator] status is now ACTIVE after event "SECURITY_DISPATCHED" from SecurityGuard-1
[DoorController] bolt engaged.
[DoorController] bolt engaged.
[DoorController] bolt engaged.
Unit is unlocked, securing and moving to locked
Unit is unlocked, securing and moving to locked
[Facilities-1] area physically and logically secured.
[CampusAlertSystem] campus-wide alert: Affected area has been secured.
[IncidentCoordinator] status is now CONTAINED after event "AREA_SECURED" from Facilities-1
[CampusAlertSystem] campus-wide alert: Security responding to Residence Block C, please remain in your rooms.
```

```
Block C locked after response: yes
Engineering door still bolted from Scenario 1: yes
Incident 2 has been reported, moving to dispatched
Incident status: Dispatched
```

```
Attempting to dispatch medic to a null destination...
Handled safely: null destination detected, medic's assignment left unchanged.
```

```
[SCENARIO 3: Escalation and System Stand Down]
Hitting remaining state transitions, unlock procedures, and coordinator branches
```

```
-- Testing Incident State Escalation --
Incident 2 units have been dispatched, marking as resolved
Incident 2 is already resolved, nothing more to do

-- Testing Zone State Escalation --

-- Testing Hardware and Logical Unlocking --
[DoorController] bolt released.

-- Testing Mediator Threat and Stand Down Broadcasts --
[IncidentCoordinator] THREAT DETECTED. Triggering response protocols.
[Medic-1] standing down.
[Facilities-1] restoring normal access.
[CampusAlertSystem] campus-wide alert: Incident resolved. Normal operations resumed.
```

```
Hitting remaining state transitions, unlock procedures, and coordinator branches
```

```
-- Testing Incident State Escalation --
Incident 2 units have been dispatched, marking as resolved
Incident 2 is already resolved, nothing more to do

-- Testing Zone State Escalation --

-- Testing Hardware and Logical Unlocking --
[DoorController] bolt released.

-- Testing Mediator Threat and Stand Down Broadcasts --
[IncidentCoordinator] THREAT DETECTED. Triggering response protocols.
[Medic-1] standing down.
[Facilities-1] restoring normal access.
[CampusAlertSystem] campus-wide alert: Incident resolved. Normal operations resumed.
[IncidentCoordinator] status is now RESOLVED after event "INCIDENT_RESOLVED" from SecurityGuard-1
```

```
-- Testing Composite Memory Management --
==1938==
==1938== HEAP SUMMARY:
==1938==    in use at exit: 0 bytes in 0 blocks
==1938== total heap usage: 82 allocs, 82 frees, 76,765 bytes allocated
==1938==
==1938== All heap blocks were freed -- no leaks are possible
==1938==
==1938== For lists of detected and suppressed errors, rerun with: -s
==1938== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
hp@DESKTOP-8KLCRG6: /mnt/c/Users/hp/OneDrive - University of Pretoria/Documents/GitHub/COS-214-Practical-5$
```

GitHub repository link and short collaboration reflection

Github link - <https://github.com/Melcodes03/COS-214-Practical-5>

The team's branching structure followed a GitHub Flow model with a shared dev branch and individual feature branches for each member namely Robert, Chelsy, and Mel. Each person committed and pushed their work on their own branch, using branch names like feature/composite-robert and feature/facade-mel to keep work separated by pattern. Each person opened a pull request into dev once their subsystem compiled and ran independently. Work was divided by design pattern across the six required patterns. These patterns were Command, Composite, Mediator, Adapter, State, and Facade. State was built first since Incident is the core object the other patterns act on, and Facade was

built last since it depends on the other three subsystems existing. Once all subsystems were merged into dev they were tested together and verified with a clean build, a Valgrind run, and a GDB investigation. Dev was then merged into main as the final step for submission.

Robert was responsible for the Command and Composite patterns. Robert implemented `CampusCommand` as the abstract command interface and `DispatchSecurityCommand`, `DispatchMedicCommand`, `SecureZoneCommand`, and `ActivateEmergencyCommand` as the concrete commands, each binding a specific receiver to a specific action. Robert also implemented `CampusComponent` as the abstract component, `CampusUnit` as the leaf representing a single room, and `CampusArea` as the composite so that locking a whole building works the same way as locking a single room. When it became clear that a room's lock status and an incident's operational status were being modelled as the same thing, Robert separated the two by building a dedicated `ZoneState` hierarchy with `LockedState`, `UnlockedState`, and `EvacuationState`, and migrated `CampusUnit` onto it so it no longer shared a state machine with `Incident`. This kept State and Composite as two clean, independent patterns rather than one overloaded class doing both jobs.

Chelsy was responsible for the Mediator and Adapter patterns. Chelsy implemented `ResponseHub` as the mediator interface and `IncidentCoordinator` as the concrete mediator, along with `ResponseUnit` as the abstract colleague and `SecurityUnit`, `MedicUnit`, `FacilitiesUnit`, and `AlertService` as the concrete colleagues that notify the coordinator instead of contacting each other directly. Chelsy also implemented `AccessPoint` as the adapter target, `DoorController` as the legacy interface with its own method names, and `LegacyDoorAccess` as the adapter translating between them. Chelsy connected the Adapter into the rest of the system by adding `secureArea()` to `FacilitiesUnit` so that securing a zone during an emergency actually engages the legacy door hardware rather than only updating a state flag. Chelsy also owned the `Makefile` and `main.cpp`, integrating all three subsystems into one buildable, runnable application.

Mel was responsible for the State and Facade patterns. Mel implemented `IncidentState` and its three concrete states, `ReportedState`, `DispatchedState`, and `ResolvedState`, along with `Incident` as the context that delegates `transition()` to whichever state is current instead of checking a status flag. This was pushed first, in small commits such as "Add `IncidentState` abstract class" and "Implement `ReportedState`," along with a manual test proving the transitions worked before anyone else built against it. Mel then implemented `EmergencyFacade`, which coordinates `DispatchSecurityCommand`, `DispatchMedicCommand`, `SecureZoneCommand`, and `ActivateEmergencyCommand` in a single `declareEmergency()` call, making `EmergencyFacade` double as the pattern's `Invoker` as well as its `Facade`. While integrating `Facade` against the other two subsystems, Mel found and reported two real bugs rather than fixing them silently: a missing return statement in `CampusUnit::isLocked()` that caused it to always report false regardless of the door's actual state, and a memory ownership issue where `CampusUnit`'s destructor deleted an `AccessPoint` that had not been heap allocated, causing a crash on exit. Mel also caught an accidental find-and-replace that had overwritten `IncidentState` with `ZoneState` inside `Incident.h` and `Incident.cpp`, and restored both files before the corruption spread further.

Across the team the six patterns were developed largely in parallel once the core interfaces on `Incident`, `ResponseUnit`, and `CampusComponent` were agreed early on, which let each subsystem be tested independently before integration. The point where the design comes together as one coherent system rather than six isolated demonstrations is `EmergencyFacade`'s `declareEmergency()` method. Here a single call dispatches security and medics through `Command`, causes those units to notify each

other through Mediator, secures a zone through Composite and Adapter together, and finally transitions the incident's own state. All of this was built collaboratively across three branches, merged through reviewed pull requests into dev, and finally pushed to main once the full system was verified for submission.

Contribution statement for each team member

Robert

Implemented the Command pattern (CampusCommand and the four concrete commands: DispatchSecurityCommand, DispatchMedicCommand, SecureZoneCommand, ActivateEmergencyCommand) and the Composite pattern (CampusComponent, CampusUnit, CampusArea). Built the separate Zone/lock state machine (ZoneState, LockedState, UnlockedState, EvacuationState) and migrated CampusUnit onto it once it was clear a room's lock status shouldn't share a state machine with an incident's status. Wrote the Dockerfile and docker-compose.yml. Drew the initial full UML class diagram and produced the sequence diagram covering Command interacting with another pattern. Wrote the design and ownership rationale section for Command and Composite.

Chelsy

Implemented the Mediator pattern (ResponseHub, IncidentCoordinator, and the concrete colleagues SecurityUnit, MedicUnit, FacilitiesUnit, AlertService) and the Adapter pattern (AccessPoint, DoorController, LegacyDoorAccess). Connected the Adapter into the rest of the system through FacilitiesUnit::secureArea(), so securing a zone actually engages the legacy door hardware. Owned the Makefile and main.cpp, integrating all three subsystems into one buildable, runnable application, and built both end-to-end demo scenarios. Wrote the short system description and produced the sequence diagram covering Facade or Adapter. Handled at least one failure/invalid-operation case in the running system.

Mel

Implemented the State pattern (IncidentState, ReportedState, DispatchedState, ResolvedState) and the Facade pattern (EmergencyFacade), pushing State first since it's the foundation the other patterns build against. Tested EmergencyFacade end to end against the other two subsystems once they were available, and found and reported two real bugs during that process rather than fixing them silently: a missing return in CampusUnit::isLocked() and a memory ownership crash in CampusUnit's destructor. Produced the GoF participant mapping table for all six patterns and the extra behavioural diagram (State diagram for Incident). Wrote the README with Docker build/run instructions, the GDB investigation and Valgrind evidence, the GitHub collaboration reflection, and this contribution statement.